

D4 Track B - API Documentation

This API manages members, savings circles, contributions, payouts, and bids. It uses an Express server and a MySQL database.

Running and testing the API

Set the database connection values in `.env`, then start the backend with `npm start`.

Base URL: `http://localhost:3000`

In Postman, set the collection variable `base_url` to this address. POST and PUT requests must use **Body** → **raw** → **JSON** and Content-Type: `application/json`.

The examples in this document explain the response format. They are not a test report. IDs, dates, and returned values depend on the database. Use the ID returned by POST when testing PUT or DELETE. Do not delete the original financial records just to test an endpoint.

Common rules

- All URL IDs must be positive integers, up to 2147483647.
- Body IDs must be JSON numbers, not quoted strings.
- PUT requires all writable fields, just like POST.
- Unknown body fields are rejected.
- GET and DELETE do not need a body. There are no query parameters on these endpoints.
- List endpoints return an array; getting one record returns an object.
- DELETE returns 204 with an empty body when it succeeds.

Code	Meaning
200	Read or update succeeded
201	A record was created
204	A record was deleted; response body is empty
400	Invalid input, ID, or JSON
403	Origin is not allowed
404	Record, reference, or route was not found
409	Duplicate data, dependent records, or a business rule prevents the operation
413	JSON body exceeds 16 KB
415	POST or PUT did not use application/json
429	Request rate limit was reached
500	Unexpected server error

Field validation returns `{"errors":["Missing or invalid name"]}`. Other errors normally return `{"error":"message"}`. Error messages depend on what failed.

Members

name: nonempty text, maximum 100 characters. email: valid email address, maximum 150 characters, unique across members.

List members

GET `http://localhost:3000/members`

Success: **200**.

```
[
  {
    "member_id": 22,
    "name": "Manual Test Member",
    "email": "manual.test@example.com"
  }
]
```

Invalid URL ID: 400. Getting a missing record by ID: 404. An empty list is returned as `[]` with 200.

Get members by ID

GET `http://localhost:3000/members/22`

Path parameter: `id` is the record's `member_id`. The example uses 22.

Success: **200**.

```
{
  "member_id": 22,
  "name": "Manual Test Member",
  "email": "manual.test@example.com"
}
```

Invalid URL ID: 400. Getting a missing record by ID: 404. An empty list is returned as `[]` with 200.

Create members

POST `http://localhost:3000/members`
Content-Type: `application/json`

```
{
  "name": "Manual Test Member",
  "email": "manual.test@example.com"
}
```

Success: **201**.

```
{
  "member_id": 22,
  "name": "Manual Test Member",
  "email": "manual.test@example.com"
}
```

Failure example: use the same method and URL with this body:

```
{
  "name": "",
  "email": "manual.test@example.com"
}
```

Response: **400**

```
{
  "errors": [
    "Missing or invalid name"
  ]
}
```

A missing record/reference can return 404. Duplicate data or an operation blocked by financial rules can return 409.

Update members

PUT http://localhost:3000/members/22
Content-Type: application/json

```
{
  "name": "Manual Test Member",
  "email": "manual.test@example.com"
}
```

Path parameter: `id` is the record's `member_id`. The example uses 22.

Success: **200**.

```
{
  "member_id": 22,
  "name": "Manual Test Member",
  "email": "manual.test@example.com"
}
```

Failure example: use the same method and URL with this body:

```
{
  "name": "",
  "email": "manual.test@example.com"
}
```

Response: **400**

```
{
  "errors": [
    "Missing or invalid name"
  ]
}
```

A missing record/reference can return 404. Duplicate data or an operation blocked by financial rules can return 409.

Delete members

DELETE http://localhost:3000/members/22

Path parameter: `id` is the record's `member_id`. The example uses 22.

Success: **204**. Empty response body.

Failure example: delete a valid positive ID that does not exist. Response: **404**

```
{
  "error": "Member not found"
}
```

Dependent records can cause 409 instead.

Circles

name: nonempty text, maximum 100 characters. contribution_amount: greater than zero, at most 99999999.99, maximum two decimal places. start_date: a valid date in YYYY-MM-DD format. Progress fields are calculated by the server and must not be sent in the body.

List circles

GET http://localhost:3000/circles

Success: **200**.

```
[
  {
    "circle_id": 2,
    "name": "Test Circle",
    "contribution_amount": "1000.00",
    "start_date": "2026-10-07",
    "member_count": 0,
    "paid_member_count": 0,
    "remaining_member_count": 0,
    "status": "pending"
  }
]
```

Invalid URL ID: 400. Getting a missing record by ID: 404. An empty list is returned as [] with 200.

Get circles by ID

GET http://localhost:3000/circles/2

Path parameter: id is the record's circle_id. The example uses 2.

Success: **200**.

```
{
  "circle_id": 2,
  "name": "Test Circle",
  "contribution_amount": "1000.00",
  "start_date": "2026-10-07",
  "member_count": 0,
  "paid_member_count": 0,
  "remaining_member_count": 0,
  "status": "pending"
}
```

Invalid URL ID: 400. Getting a missing record by ID: 404. An empty list is returned as [] with 200.

Create circles

POST http://localhost:3000/circles

Content-Type: application/json

```
{
  "name": "Test Circle",
  "contribution_amount": 1000,
  "start_date": "2026-10-07"
}
```

Success: **201**.

```
{
  "circle_id": 2,
  "name": "Test Circle",
  "contribution_amount": "1000.00",
  "start_date": "2026-10-07",
  "member_count": 0,
  "paid_member_count": 0,
  "remaining_member_count": 0,
  "status": "pending"
}
```

Failure example: use the same method and URL with this body:

```
{
  "name": "Test Circle",
  "contribution_amount": 0,
  "start_date": "2026-10-07"
}
```

Response: **400**

```
{
  "errors": [
    "Missing or invalid contribution_amount"
  ]
}
```

A missing record/reference can return 404. Duplicate data or an operation blocked by financial rules can return 409.

Update circles

PUT <http://localhost:3000/circles/2>
Content-Type: application/json

```
{
  "name": "Test Circle",
  "contribution_amount": 1000,
  "start_date": "2026-10-07"
}
```

Path parameter: `id` is the record's `circle_id`. The example uses 2.

Success: **200**.

```
{
  "circle_id": 2,
  "name": "Test Circle",
  "contribution_amount": "1000.00",
  "start_date": "2026-10-07",
  "member_count": 0,
  "paid_member_count": 0,
  "remaining_member_count": 0,
  "status": "pending"
}
```

Failure example: use the same method and URL with this body:

```
{
  "name": "Test Circle",
  "contribution_amount": 0,
  "start_date": "2026-10-07"
}
```

Response: **400**

```
{
  "errors": [
    "Missing or invalid contribution_amount"
  ]
}
```

A missing record/reference can return 404. Duplicate data or an operation blocked by financial rules can return 409.

Delete circles

DELETE http://localhost:3000/circles/2

Path parameter: `id` is the record's `circle_id`. The example uses 2.

Success: **204**. Empty response body.

Failure example: delete a valid positive ID that does not exist. Response: **404**

```
{
  "error": "Circle not found"
}
```

Dependent records can cause 409 instead.

Contributions

`member_id` and `cycle_id`: positive integer IDs. The member must belong to the circle of the cycle. Only one contribution per member per cycle is allowed. There is no amount field; the amount comes from the circle.

Contributions cannot be changed or deleted after the cycle has a payout. Creating or updating a contribution can trigger automatic settlement if the cycle is ready. The sample member/cycle pair must be unused for a successful POST; the original seed may already contain it.

List contributions

GET http://localhost:3000/contributions

Success: **200**.

```
[
  {
    "contribution_id": 61,
    "member_id": 1,
    "cycle_id": 6
  }
]
```

Invalid URL ID: 400. Getting a missing record by ID: 404. An empty list is returned as `[]` with 200.

Get contributions by ID

GET http://localhost:3000/contributions/61

Path parameter: `id` is the record's `contribution_id`. The example uses 61.

Success: **200**.

```
{
  "contribution_id": 61,
  "member_id": 1,
  "cycle_id": 6
}
```

Invalid URL ID: 400. Getting a missing record by ID: 404. An empty list is returned as [] with 200.

Create contributions

POST http://localhost:3000/contributions
Content-Type: application/json

```
{
  "member_id": 1,
  "cycle_id": 6
}
```

Success: **201**.

```
{
  "contribution_id": 61,
  "member_id": 1,
  "cycle_id": 6
}
```

Failure example: use the same method and URL with this body:

```
{
  "member_id": "abc",
  "cycle_id": 6
}
```

Response: **400**

```
{
  "errors": [
    "Missing or invalid member_id"
  ]
}
```

A missing record/reference can return 404. Duplicate data or an operation blocked by financial rules can return 409.

Update contributions

PUT http://localhost:3000/contributions/61
Content-Type: application/json

```
{
  "member_id": 1,
  "cycle_id": 6
}
```

Path parameter: id is the record's contribution_id. The example uses 61.

Success: **200**.

```
{
  "contribution_id": 61,
  "member_id": 1,
  "cycle_id": 6
}
```

Failure example: use the same method and URL with this body:

```
{
  "member_id": "abc",
  "cycle_id": 6
}
```

Response: **400**

```
{
  "errors": [
    "Missing or invalid member_id"
  ]
}
```

A missing record/reference can return 404. Duplicate data or an operation blocked by financial rules can return 409.

Delete contributions

DELETE <http://localhost:3000/contributions/61>

Path parameter: `id` is the record's `contribution_id`. The example uses 61.

Success: **204**. Empty response body.

Failure example: delete a valid positive ID that does not exist. Response: **404**

```
{
  "error": "Contribution not found"
}
```

Dependent records can cause 409 instead.

Payouts

`cycle_id` and `member_id`: positive integer IDs. `paid_at`: a valid timestamp in YYYY-MM-DD HH:mm:ss format. Payouts use `cycle_id` as their primary key; there is no `payout_id`. For PUT, `cycle_id` in the body must match the URL ID.

Normal open-cycle payouts are created by automatic bid settlement. Manual POST is an administrative operation for a closed, fully funded cycle without an existing payout. The recipient must be the highest eligible bidder, or the last unpaid member. A previous winner cannot receive another payout in the same circle. Changing a recipient with existing discount shares is blocked. Manual POST does not create discount shares. Deletion may be blocked by dependent discount shares.

List payouts

GET <http://localhost:3000/payouts>

Success: **200**.

```
[
  {
    "cycle_id": 6,
    "member_id": 1,
    "paid_at": "2026-10-07 12:00:00"
  }
]
```

Invalid URL ID: 400. Getting a missing record by ID: 404. An empty list is returned as `[]` with 200.

Get payouts by ID

GET <http://localhost:3000/payouts/6>

Path parameter: `id` is the record's `cycle_id`. The example uses 6.

Success: **200**.

```
{
  "cycle_id": 6,
  "member_id": 1,
  "paid_at": "2026-10-07 12:00:00"
}
```

Invalid URL ID: 400. Getting a missing record by ID: 404. An empty list is returned as [] with 200.

Create payouts

POST http://localhost:3000/payouts
Content-Type: application/json

```
{
  "cycle_id": 6,
  "member_id": 1,
  "paid_at": "2026-10-07 12:00:00"
}
```

Success: **201**.

```
{
  "cycle_id": 6,
  "member_id": 1,
  "paid_at": "2026-10-07 12:00:00"
}
```

Failure example: use the same method and URL with this body:

```
{
  "cycle_id": 6,
  "member_id": 1,
  "paid_at": "invalid"
}
```

Response: **400**

```
{
  "errors": [
    "Missing or invalid paid_at"
  ]
}
```

A missing record/reference can return 404. Duplicate data or an operation blocked by financial rules can return 409.

Update payouts

PUT http://localhost:3000/payouts/6
Content-Type: application/json

```
{
  "cycle_id": 6,
  "member_id": 1,
  "paid_at": "2026-10-07 12:00:00"
}
```

Path parameter: id is the record's cycle_id. The example uses 6.

Success: **200**.

```
{
  "cycle_id": 6,
  "member_id": 1,
  "paid_at": "2026-10-07 12:00:00"
}
```

Failure example: use the same method and URL with this body:

```
{
  "cycle_id": 6,
  "member_id": 1,
  "paid_at": "invalid"
}
```

Response: **400**

```
{
  "errors": [
    "Missing or invalid paid_at"
  ]
}
```

A missing record/reference can return 404. Duplicate data or an operation blocked by financial rules can return 409.

Delete payouts

DELETE `http://localhost:3000/payouts/6`

Path parameter: `id` is the record's `cycle_id`. The example uses 6.

Success: **204**. Empty response body.

Failure example: delete a valid positive ID that does not exist. Response: **404**

```
{
  "error": "Payout not found"
}
```

Dependent records can cause 409 instead.

Circle members and cycles

Get members of a circle

GET `http://localhost:3000/circles/1/members`

Path parameter: `id` is the circle ID. No body.

Success: **200**, an array of member objects containing `member_id`, `name`, `email`, and `joined_on`.

```
[
  {
    "member_id": 1,
    "name": "Ahmed Ali",
    "email": "ahmed@example.com",
    "joined_on": "2026-01-01"
  }
]
```

Get cycles of a circle

GET `http://localhost:3000/circles/1/cycles`

Path parameter: `id` is the circle ID. No body.

Success: **200**:

```
[
  {
    "cycle_id": 6,
    "seq_no": 6,
    "status": "open"
  }
]
```

For both endpoints, an invalid ID returns 400; a missing circle returns 404. A circle with no members or cycles returns an empty array.

Circle progress uses pending when there are no members, active when some members still need a payout, and closed when all current members have received one. `remaining_member_count` equals `member_count` minus `paid_member_count`.

Bids

Submit a bid

POST `http://localhost:3000/cycles/6/bids`
Content-Type: `application/json`

```
{
  "member_id": 1,
  "discount_rate": 10
}
```

Path parameter: `id` is the cycle ID. `discount_rate` is a percentage greater than zero and at most 100, with up to two decimal places. The member must belong to the circle and must not have received a previous payout there. The cycle must be open, and each member can bid only once per cycle.

Success: **201**:

```
{
  "bid_id": 1,
  "member_id": 1,
  "cycle_id": 6,
  "discount_rate": "10.00",
  "cycle_status": "open"
}
```

`cycle_status` can be closed when this bid completes the requirements for settlement.

Failure example: send the same request with this body:

```
{"member_id": 1, "discount_rate": -5}
```

Response: **400**:

```
{"errors": ["Missing or invalid discount_rate"]}
```

A missing cycle/member returns 404. A closed cycle, duplicate bid, previous winner, or final member trying to bid returns 409.

Get bid ranking

GET `http://localhost:3000/cycles/6/bids/ranking`

Path parameter: `id` is the cycle ID. No body.

Success: **200**:

```
[
  {"rank": 1, "bid_id": 1, "member_id": 1, "cycle_id": 6, "discount_rate": "10.00",
  "name": "Ahmed Ali"}
]
```

Bids are sorted by highest discount rate first. Equal rates are sorted by lowest bid ID first. No bids returns `[]`. An invalid ID returns 400; a missing cycle returns 404.

Automatic settlement

When every eligible member has submitted a bid and every circle member has contributed to the cycle, the server creates the payout, saves discount shares, and closes the cycle in one transaction. The highest eligible bidder wins. The discount is divided among all other circle members, including earlier winners. Small rounding differences are distributed in cents by member ID.

When only one unpaid member remains, that member receives the full pot after the cycle is fully funded. No bid or discount shares are needed. The backend also checks ready open cycles at startup and periodically; the default interval is 30 seconds. It does not create the next cycle automatically.

Validation and security

The backend validates required fields and rejects unknown fields. Database queries use parameters for input values. CORS allows configured origins only; the default is `http://localhost:5173`. Requests without an Origin header, such as normal Postman requests, are allowed.

The default rate limit is 100 requests per IP per 60 seconds. These settings can be changed in `.env`. JSON bodies are limited to 16 KB. Unexpected server errors return a generic message instead of database details. This API does not include login or user authentication.

A simple test sequence

1. GET `/members` and check for 200.
2. POST `/members` with a fresh email and check for 201. Copy the returned member ID.
3. GET `/members/{new_id}` and check the saved data.
4. PUT `/members/{new_id}` with all required fields and check for 200. Keep the same email or use another unused email.
5. DELETE `/members/{new_id}` and check for 204 with an empty body.
6. GET the deleted ID and check for 404.
7. Submit an invalid body and check for 400. Reusing another member's email should return 409.

Use separate test records for circle writes. Financial writes need valid memberships and cycles, so a blocked request can be correct behavior. Creating a circle or member alone does not create memberships or cycles. Set up suitable financial test data before testing contributions, payouts, and automatic bid closing.

Save actual test results separately in the validation/security test log. Saved example responses in Postman are documentation, not proof that a request was executed.